Cairo University - Faculty Of Engineering
Computer Engineering Department
CMP4007 Applied Data Science - Spring 2026

Project Final Report

Team 9

Eng. Eman Hosam

#	Name	ID	Section	Bench No.
1	Ahmed Hamdy	9220032	1	4
2	Ahmed Aladdin	9220065	1	5
3	Asmaa Mohamed	9220154	1	29
4	Shehab Khaled	9220393	1	38

Contents

1 Problem Definition & Business Context

1.1 Business Problem

Used car dealerships and online marketplaces receive large volumes of vehicle listings and trade-ins. Manually assigning each vehicle into a pricing/value tier is time-consuming and inconsistent across staff and branches. This project aims to automate the classification of incoming used cars into value tiers using historical market data.

1.2 Stakeholders

- Primary: Used Car Dealerships (pricing and inventory teams)
- Secondary: Online Marketplaces (listing recommendations) and Insurance Underwriters (risk and valuation support)

1.3 Problem Formulation

This problem is formulated as a **classification task**, where the goal is to predict the **price/value tier** of a used car listing.

Tier Definition: Cars are grouped into **Budget**, **Mid-Range**, and **Luxury** tiers based on price.

1.4 Justification for Classification

- The target is a discrete set of tiers (Budget / Mid-Range / Luxury)
- Tier outputs support operational decisions (pricing, procurement, and marketing)
- Classification enables threshold-based policies and consistent segmentation

1.5 Business Impact

- Faster and more consistent inventory categorization
- Improved pricing strategy and margin optimization
- Better customer targeting (budget buyers vs. premium buyers)

2 Data Acquisition & Integration

2.1 Sources Description

2.1.1 Primary Source (Kaggle Dataset)

Source: Kaggle — *Uncovering Factors That Affect Used Car Prices*
URL: https://www.kaggle.com/datasets/thedevastator/uncovering-factors-that-affect-used-car-prices

autos.csv

Main structured dataset (CSV). Each row represents a used car listing with attributes such as price, brand, model, registration year, mileage, and fuel type.

2.1.2 Secondary Source (Web Scraping)

Source: AutoScout24
URL: `https://www.autoscout24.com`

AutoScout24 Listings

Scraped used car listings to increase coverage and validate pricing patterns.

2.2 Data Acquisition

- Kaggle dataset downloaded as CSV file(s) (e.g., `autos.csv`).
- AutoScout24 listings collected via web scraping using Python (e.g., `Scrapy` + `Scrapy-Splash`).

2.3 Data Integration Strategy

- **Vertical merge** the Kaggle dataset with scraped AutoScout24 listings by aligning both sources to a unified schema.
- Standardize fields and resolve naming differences (e.g., `kilometer` vs `mileage`).
- Deduplicate near-identical listings using matching keys such as **brand**, **model**, **kilometer**, **yearOfRegistration**, and **power**.
- Merge strategy is union rows after schema alignment.

Challenges:

- Inconsistent naming and category sets across sources
- Translating the Kaggle dataset from German to English
- Missing or noisy scraped fields due to unstructured HTML
- Potential duplicates across sources

2.4 Feature Documentation (Core Fields)

The accepted features to be used after the merging and removal of redundant features:

#	Feature	Type	Description
1	seller	Categorical	seller type (business/private)
2	vehicleType	Categorical	body type
3	yearOfRegistration	Numeric	Year the car was registered
4	gearbox	Categorical	Type of gearbox (manual or automatic)
5	power	Numeric	Power of the car in Horse Power
6	model	Categorical	Model of the car
7	brand	Categorical	Brand of the car
8	kilometer	Numeric	Kilometers the car has been driven
9	fuelType	Categorical	Type of fuel (e.g. diesel, petrol, etc.)
10	dataSource	Categorical	Source of the data (kaggle/crawled)
11	price	Numeric	Price of the car
12	price_tier	Categorical	Target tier derived from price

Note: After deriving the tier label from `price`, the raw `price` feature will not be used as an input feature to avoid target leakage.

3 Data Validation & Documentation

3.1 Dataset Validation Pipeline

Step	Description	Outcome
1	Schema Validation	Data type report
2	Completeness Check	Missing value report
3	Duplicate Detection	Duplicate count
4	Descriptive Statistics	Summary stats
5	Range & Outlier Checks	Violation report
6	Categorical Analysis	Category distribution
7	Distribution Checks	Distribution plots
8	Outlier Analysis	IQR & Z-Score & Isolation Forest Outliers

3.2 Dataset Validation Report

Metric	Value
Total checks	9
Checks Passed	1
Checks FAILed	8
Success Rate	11.1%
Full-row duplicates	57,324 (15.33%)

3.2.1 Completeness - Missing Values Analysis

Column	Missing %	Status
brand	0.00%	PASS
model	5.48%	FAIL (> 5%)
vehicleType	10.12%	FAIL (> 5%)
power	0.00%	PASS
gearbox	5.40%	FAIL (> 5%)
kilometer	0.01%	PASS
fuelType	8.93%	FAIL (> 5%)
yearOfRegistration	0.02%	PASS
seller	0.00%	PASS
dataSource	0.00%	PASS
price	0.00%	PASS
price_tier	0.00%	PASS

3.2.2 Descriptive Statistics (Numeric Columns)

Column	Mean	Std	Mode	Min	Q1	Median	Q3	Max
yearOfRegistration	2004.68	92.57	2000	1000	1999	2003	2008	9999
power	115.98	191.77	0	0	70	105	150	20,000
kilometer	125,298.65	40,548.44	150,000	0	100,000	150,000	150,000	999,999
price	25,472.31	5,262,458.17	0	0	1,692.39	4,400.21	10,742.99	3.16 B

3.2.3 Distribution Checks

Column	Skewness	Skewness Label	Kurtosis	Kurtosis Label
yearOfRegistration	72.35	Positive Skew	5702.44	Leptokurtic
price	580.00	Positive Skew	347759.38	Leptokurtic
kilometer	-1.48	Negative Skew	1.75	Leptokurtic
power	58.14	Positive Skew	4428.10	Leptokurtic

3.2.4 Consistency - Schema & Data Types

Column	Detected Type	Expected Type	Match
yearOfRegistration	float64	int64	No
model	str	str	Yes
power	float64	float64	Yes
kilometer	float64	float64	Yes
price	float64	int64	No
seller	object	str	No
gearbox	str	str	Yes
vehicleType	str	str	Yes
fuelType	str	str	Yes
brand	str	str	Yes
dataSource	object	str	No
price_tier	object	str	No

3.2.5 Validity Checks

Column	Rule	Violations	Status
yearOfRegistration	[1900 - 2026]	182 (68 below, 114 above)	FAIL
kilometer	[0 - 300,000]	13 (above max)	FAIL
power	[5 - 5,000]	40,988 (40,903 below, 85 above)	FAIL
price	[500 - 5,000,000]	26,273 (26,219 below, 54 above)	FAIL

3.2.6 Uniqueness - Duplicate Detection

- Full-row duplicates detected: **57,324 rows (15.33%)**.

3.2.7 Statistical Outlier Detection

Column	Z-Score Outliers ($\|z\| > 3.0$)	Z-Score %	IQR Outliers	IQR %	Status
price	40	0.01%	27,980	7.48%	FAIL
power	385	0.10%	11,035	2.95%	FAIL
kilometer	290	0.08%	15,397	4.12%	FAIL
yearOfRegistration	173	0.05%	8,289	2.22%	FAIL

Multivariate Outlier Detection (Isolation Forest):

- Detected **44,875 outliers (12.00%)** across combined numeric features (`price`, `power`, `kilometer`, `yearOfRegistration`).

3.3 Quality Issues Summary

- **Schema Issue:** `yearOfRegistration` and `price` are formatted as `float64` instead of the expected `int64`.
- **Missing Data:** Significant missing values across `vehicleType` (10.12%), `fuelType` (8.93%), `model` (5.48%), and `gearbox` (5.40%).
- **Duplicate Records:** High proportion of exact full-row duplicates detected (15.33%).
- **Invalid Ranges:** Massive positive skew in `price` (max 3.16B) and `power` (max 20k) indicating corrupted entries. `power` heavily violates minimum range boundaries (40,903 values below 5). `price` also violates minimum boundaries (26,219 below 500).
- **Isolation Forest:** flagged 44,875 anomalous records (12.00% of the dataset).

3.4 Target Variable Definition

Target Definition

PRICE_TIER = value tier derived from listing `price`.

Tier Construction:

Tier	Label	Price Range	Business Meaning	Business Interpretation
0	Budget	< \$5,000	High-mileage or older cars	Entry-level pricing segment
1	Mid-Range	\$5,000 - \$14,999	Recent-model mid-size cars	Mainstream segment
2	Luxury	≥ \$15,000	Luxury, electric, or new cars	Premium segment

Class Distribution [45.17% Budget, 34.42% Mid-Range, 20.41% Luxury]

4 Preprocessing, Transformation & Engineering Data

4.1 Preprocessing Pipeline

Step	Description	Tools	Outcome
1	Data Cleaning	Pandas	Cleaned dataset (259,092 rows)
2	Outlier Detection	IQR, Fixed Bounds	Outlier report
3	Feature Transformation	Scikit-learn	Transformed features
4	Feature Engineering	Custom code	New features
5	Feature Selection	Scikit-learn	Selected features
6	Train-Validation-Test Split	Scikit-learn	Data splits
7	Data Balancing	SMOTE, Undersampling	Balanced training set

4.2 Cleaning Steps & Justification

- **Remove invalid rows (58,245 removed):** Removed entries violating logical domain constraints to ensure data quality.
  - `yearOfRegistration` outside [1900, 2026].
  - `kilometer` outside [0, 300,000].
  - `power` outside [5, 3000].
  - `price` outside [500, 3,000,000].
- **Duplicate Handling:** Dropped 54,495 full-row duplicated records to prevent data leakage.
- **Missing Value Imputation:**
  - Replaced uninformative placeholders (e.g., "N/A", "unknown", "-") with `NaN` prior to imputation.
  - Imputed missing categorical values using the group-wise mode based on the car's `brand`.
- **Standardization:** Cleaned and standardized categorical labels (e.g., combining German/English aliases for fuel types, vehicle types, and brands).

4.3 Outlier Detection & Handling

Column	Detection Method	Handling Strategy	Outcome
power	IQR	Cap bounds to [5.0, 252.0]	12,530 outliers capped
price	Fixed Ranges	Drop rows outside [500, 3,000,000]	Extreme values removed
kilometer	IQR	Cap bounds to [0.0, 225,000]	66 outliers capped
yearOfRegistration	IQR	Cap bounds to [1900, 2023]	553 outliers capped

4.4 Feature Transformation & Justification

- **Numeric Imputation (median):** Robust to skewed distributions and outliers common in price.
- **Categorical Imputation (most_frequent):** Mode imputation provides a stable default without introducing new categories.
- **Scaling (StandardScaler):** Improves optimization and comparability for linear models.
- **One-Hot Encoding:** Strong baseline for mixed-type tabular data reducing collinearity.
- **Frequency Encoding for brand/model:** Prevents dimensionality explosion from high-cardinality variables while preserving relative frequency information.

4.5 Feature Engineering

In addition to the core fields, we have engineered the following features:

- **vehicleAge**: Calculated as (current year − year of registration) to capture physical depreciation.
- **kmPerYear**: Normalizes the mileage by the age of the vehicle to represent usage intensity.
- **km_power_ratio**: A proxy for the interaction between usage and engine power to capture non-linear relationships.
- **log_kilometer** & **log_power**: Logarithmic transformations to compress right-skewed outliers.
- **power_per_age**: Represents performance relative to the age of the vehicle.
- **is_vintage**: A binary indicator for vehicles exceeding the standard vintage age threshold.
- **is_automatic**: A direct binary feature extracted from the gearbox field.

4.6 Feature Selection & Justification

- We applied a model-based feature selection method using a Random Forest classifier to identify the most predictive features while accounting for non-linear relationships and interactions.
- The threshold was set to the median importance to retain features that contribute above-average predictive power.
- This method is more robust than simple variance or correlation-based filters, especially in the presence of engineered features that may have complex relationships with the target variable.
- Outcome: The selection process resulted in the removal of low-importance features, reducing noise and improving model generalization.

4.7 Train-Validation-Test Split Procedure

- **Train-Test Split:** An 80/20 stratified split was performed to ensure that the class distribution of the target variable is preserved across both sets, which is crucial for imbalanced classification tasks.
- **Cross-Validation:** A 5-fold stratified cross-validation strategy was implemented on the training set to provide robust estimates of model performance and to mitigate variance in
- the presence of class imbalance, ensuring that each fold maintains the same proportion of classes as the overall dataset.

4.8 Data Balancing Strategy

We experimented with the following resampling techniques to address class imbalance:

1. **None (Baseline):** No resampling applied to establish a baseline for the bias-variance trade-off.

2. **SMOTE (Synthetic Minority Oversampling Technique):** Generates synthetic samples for the minority class to improve recall, particularly for the luxury tier, while being cautious of potential overfitting.

3. **Undersampling (RandomUnderSampler):** Randomly removes samples from the majority class to balance the dataset, which can speed up training but may lead to information loss and higher variance, especially in the budget tier.

An evaluation of the impact of each technique on both global performance metrics (e.g., F1 score) and business-specific metrics (e.g., luxury recall) was conducted to determine the optimal approach.

These are the results of the resampling techniques on the final model performance:

Technique	Outcome
None	Highest overall F1 and lowest severe error rate.
SMOTE	Increased luxury recall but hurt global accuracy.
Undersampling	Highest luxury recall but highest error rate.

4.9 Preprocessing Summary

- Cleaned dataset by removing invalid rows and handling duplicates
- Detected and handled outliers in power, price, and kilometers
- Imputed missing numeric values with the median and categorical values with the mode.
- Applied frequency encoding for high-cardinality columns and one-hot encoding for the remaining.
- Scaled numeric features using a StandardScaler to stabilize model optimization.
- Engineered new features to capture domain-specific insights and interactions.
- Selected features based on model importance to improve generalization.
- Evaluated multiple data balancing techniques to address class imbalance while monitoring the impact on both global and business-specific performance metrics.

4.10 Dataset After and Before Preprocessing

Dataset	Before Preprocessing	After Preprocessing
Total rows	371,528	350,000
Total columns	21	15
Missing values	194,786	0
Outliers	Detected in power, price, kilometers	Handled by capping
Class distribution	54.81% Budget, 37.19% Mid-Range, 7.99% Luxury	Balanced to 33.3% each

5 Exploratory Data Analysis (EDA)

5.1 EDA Dashboard

6 Model Development

We evaluated multiple model architectures to address the imbalanced 3-class target variable (budget, mid-range, luxury).

- **Baseline**: A `DummyClassifier` was used to establish the minimum expected performance.
- **Linear Models**: `LogisticRegression` and `LinearSVC` were tested as strong, fast baselines for tabular data.
- **Single Tree**: `DecisionTreeClassifier` was included as an interpretable non-linear baseline.
- **Ensembles**: `RandomForestClassifier` and `ExtraTreesClassifier` were utilized to natively capture non-linearities and feature interactions.
- **Gradient Boosting**: `XGBClassifier` was selected as the state-of-the-art candidate for handling complex tabular patterns.

Hyperparameter tuning was executed using `RandomizedSearchCV` optimized for the macro F1 score.

7 Experiment Tracking with MLflow

All modeling experiments, parameters, and metrics were tracked utilizing MLflow. We logged standard metrics such as Macro F1 and Balanced Accuracy, alongside specific business metrics including Luxury Recall and Severe Misclassification Rate. Visual tracking artifacts, such as confusion matrices and a comprehensive model comparison dashboard, were recorded to support stakeholder review and error analysis.

8 Testing

Testing focused on assessing model generalizability and detecting overfitting between the cross-validation folds and the held-out test set.

- **Generalization**: Across all model variations, validation F1 and test F1 scores were observed to be very close, indicating no major overfitting.
- **Top Performer Stability**: The standard XGBoost model yielded a Validation F1 of 0.8591 and a nearly identical Test F1 of 0.8590.
- **Underfitting Detection**: The Logistic Regression models demonstrated significant underfitting, scoring a low F1 of approximately 0.66 and a high severe error rate.

9 Results & Evaluation

The final evaluation concluded that the **XGBoost (no resampling)** model achieved the best overall trade-off among accuracy, stability, and business safety.

9.1 Model Comparison

Model	Test F1	Test Balanced Acc	Luxury Recall	Severe Error Rate
XGBoost (none)	**0.8590**	0.8566	0.8611	**0.0022**
XGBoost (smote)	0.8560	**0.8580**	0.8857	0.0025
XGBoost (undersample)	0.8529	0.8586	**0.9053**	0.0028

9.2 Error Analysis

The primary source of error was confusion between the mid-range and luxury tiers due to overlapping feature distributions. Importantly, the model almost never confused the budget and luxury tiers. This severe misclassification rate was kept to approximately 0.22%, confirming the model is highly safe for automated business pricing decisions.

9.3 Classification Report (Best Model)

The standard XGBoost model achieved an overall accuracy of 86% on a test support size of 51,819.

- **Budget (Class 0)**: Precision 0.89, Recall 0.91, F1-Score 0.90.
- **Mid-Range (Class 1)**: Precision 0.80, Recall 0.80, F1-Score 0.80.
- **Luxury (Class 2)**: Precision 0.89, Recall 0.86, F1-Score 0.87.

10 Code Automation

11 Continous Integration

12 Deployment & Monitoring

13 Interactive Dashboard

14 Limitations & Future Work

14.1 Limitations

We acknowledge the following limitations in our current approach:

- Data quality issues such as missing values and potential outliers may impact model performance.
- Class imbalance, even after balancing techniques, may still affect the model's generlization.
- The model may not capture all relevant features influencing car pricing, such as regional market differences or economic factors.
- The current model is based on historical data and needs retraining on future market changes.

14.2 Future Work

Future work could include:

- Investigating the impact of external factors (e.g., economic conditions, seasonal trends) on car pricing and incorporating them into the model.
- Exploring advanced modeling techniques such as ensemble methods or deep learning architectures
- Evaluating model performance across different market segments and regions to ensure robustness.
- Conducting a more thorough error analysis to identify specific areas for improvement.
- Implementing a feedback loop from the deployed model to continuously improve performance based on real-world data and user interactions.

15 Team Contributions

Task	Members
Data Acquisition	Ahmed Aladdin
Data Validation	Ahmed Hamdy
Preprocessing	Ahmed Hamdy
Feature Engineering & Selection	Asmaa Mohamed
EDA & Visualization	Ahmed Aladdin
Model Development	Asmaa Mohamed
Experiment Tracking	Asmaa Mohamed
Testing	Ahmed Hamdy & Asmaa Mohamed
Results & Evaluation	Asmaa Mohamed
Code Automation & CI	Ahmed Hamdy & Ahmed Aladdin
Deployment & Dashboard	Ahmed Hamdy & Ahmed Aladdin